

API de gestion de parcelles cadastrales

Hassen Mhenni

6 août 2026

- 1 **PostGIS** — **base de données du projet** extension de postgres qui ajoute le type `geometry`, les fonctions `ST_*` et l'index `GIST`.
- 2 **GeoDjango** — **l'ORM**, dans sa version géographique. Django seul ne sait pas ce qu'est une colonne `geometry` comme `PolygonField` devient une vraie colonne géométrique, et un filtre comme `geom__touches=...` part en `ST_Touches` côté base.
- 3 **Django REST Framework** — **le cycle HTTP** : lire le corps JSON, valider les champs, renvoyer la response et paginer les listes.
- 4 **djangorestframework-gis** — **GeoJSON** ↔ **GEOSGeometry**, dans les deux sens..

Un fichier, une responsabilité

```
parcelles-api/  
|- db/schema.sql          # table, index GIST, contrainte  
|- data/parcelles.csv     # 8 521 parcelles  
|- config/  
|   '- settings.py       # backend postgis, DRF  
|- parcelles/            # l'application  
|   |- models.py         # la table  
|   |- geo.py            # func pour la geo  
|   |- serializers.py     # GeoJSON <-> Parcelle  
|   |- views.py          # cycle HTTP, filtres de zone  
|   |- urls.py           # routes  
|   |- pagination.py     # pagination  
|   |- exceptions.py     # 409 conflict  
|   |- admin.py          # admin page  
|   '- management/commands/  
|       '- init_db.py    # extension + schema + donnees  
'- .github/workflows/lint.yml # flake8 dans le githubcu
```

La table Parcelle

```
-- db/schema.sql
CREATE TABLE parcelle (
  id          bigserial PRIMARY KEY,
  code_insee  text NOT NULL,
  prefixe     text NOT NULL,
  section     text NOT NULL,
  numero      text NOT NULL,
  geom        geometry(Polygon, 4326) NOT NULL,
  min_lon     double precision NOT NULL,
  min_lat     double precision NOT NULL,
  max_lon     double precision NOT NULL,
  max_lat     double precision NOT NULL,
  surface_m2  double precision,
  created_at  timestampz NOT NULL DEFAULT now(),
  updated_at  timestampz NOT NULL DEFAULT now(),
  CONSTRAINT parcelle_identifiant_unique
    UNIQUE (code_insee, prefixe, section, numero)
);

CREATE INDEX parcelle_geom_gist
  ON parcelle USING GIST (geom);
```

Une management custom command

```
parcelles/management/commands/init_db.py
```

```
python manage.py init_db          # extension + schema + CSV  
python manage.py init_db --reset  # repartir d'une table vide
```

Le script importe `data/parcelles.csv` puis exécute `db/schema.sql` (crée la table) .

- `bulk_create(batch_size=1000).`
- `transaction.atomic()` .

Le modèle Django : la classe Parcelle

- **Un modèle** = une classe qui représente une table : un attribut par colonne.
- **class Meta** — réglages de table.
 - `managed=False` — pas de migration.
 - `db_table` — nom de table.
 - `verbose_name(_plural)` — nom dans l'admin.
 - `constraints` — unicité redéclarée ici.
- **Bbox** : `save()` redéfinit — lit `geom.extent` et remplit les 4 colonnes avant d'enregistrer.
- `geom.extent` — bbox du polygone.

Le GeoJSON (RFC 7946)

```
{
  "type": "Feature",
  "id": 4211,
  "bbox": [1.4451, 43.6042,
           1.4462, 43.6051],
  "geometry": {"type": "Polygon",
               "coordinates": [[ .. ]]},
  "properties": {
    "code_insee": "31555",
    "prefixe": "806",
    "section": "AB",
    "numero": "139",
    "surface_m2": 15774.0,
    "created_at": "...",
    "updated_at": "..."
  }
}
```

- **Même forme** en entrée et en sortie : le corps d'un GET se rejoue en POST.
- **RFC 7946** — type, geometry, properties ; id et bbox facultatifs, ici toujours remplis.
- id, bbox, surface_m2 — calculés, ignorés en entrée.
- Une liste → FeatureCollection paginée.

Sortie : to_representation

- `ModelSerializer` — les champs viennent du modèle.
- Le champ `géo` : `GEOSGeometry` → dict GeoJSON (DRF seul ne sait pas).
- La `surface` : `surface_m2` est calculée, jamais lue telle quelle.
 - `_area()` — contenance officielle si elle existe, sinon la surface annotée par PostGIS [slide 19].
 - L'annotation est un objet `Area`, pas un nombre — `.sq_m` le convertit en m^2 .
 - `round(..., 1)` — `15773.98431276321` → **15774.0** : sous le décimètre, c'est du bruit GPS.
- **Création du GeoJSON** : `type`, `id (pk)`, `bbox (le modèle)`, `geometry`, `properties`.

Entrée : to_internal_value

```
{ "type": "Feature",  
  "geometry": {...},  
  "properties": { "code_insee": .. } }  
-> { "geometry": {...},  
      "code_insee": "31555",  
      "section": "AB" }
```

- Refus net : corps qui n'est pas un objet, type $\neq$ Feature, properties mal formé → 400.
- `_normalize()`
 - `strip()` — partout.
 - `upper()` — sur section.
- Un regex par champ d'identifiant (31555, 806, AB, 139).

- `validate_geometry()` — appelé par DRF pour le seul champ `geometry` : polygone fermé, non auto-intersectant [slide 20].
- `validate()` — appelé ensuite, sur **l'objet entier** : c'est là que vivent les deux règles métier.
 - Identifiant déjà pris → 409.
 - Chevauchement avec une voisine → 409 [slide 21].
- `update()` — géométrie modifiée $\Rightarrow$ `surface_m2 = None`, une contenance héritée mentirait en utilisant `geo.same_geometry` [slide 28].

Les 8 routes de l'API

Méthode	Route	Classe	Ce que ça fait
GET	/health	HealthView	version PostGIS + nb de parcelles
GET POST	/parcelles /parcelles	ParcelleListCreateView	liste paginée + filtres crée une parcelle
GET PUT PATCH DELETE	/parcelles/{id} /parcelles/{id} /parcelles/{id} /parcelles/{id}	ParcelleDetailView	le détail d'une parcelle remplacement complet modification partielle suppression
GET	/parcelles/{id}/voisines	ParcelleNeighboursView	les parcelles qui la touchent

```
-- GET /health
SELECT PostGIS_Version();           -- curseur SQL brut
SELECT COUNT(*) FROM parcelle;      -- Parcelle.objects.count()

{"status": "ok", "postgis": "3.4 USE_GEOS=1 ...", "parcelles": 8521}
```

ParcelleListView

generics.ListCreateAPIView = GET liste + POST crée, sur la même URL /parcelles. On fournit `serializer_class` et `get_queryset()` ; DRF fait le reste.

Le queryset de départ est `geo.parcelles_with_area()` [slide 19], puis on empile les filtres d'identifiant (`code_insee`, `prefixe`, `section`, `numero`), puis le filtre de zone.

```
-- GET /parcelles?code_insee=31555&section=ab&page=2
SELECT id, code_insee, ..., geom,
       ST_Area(CAST(geom AS geography)) AS computed_area
FROM parcelle
WHERE code_insee = '31555' AND section = 'AB'    -- 'ab' -> upper()
ORDER BY id
LIMIT 200 OFFSET 200;                          -- la pagination DRF
```

- La surface est calculée par la base, dans la même requête ça évite des aller retour.

_filter_by_zone — ?bbox= ou ?zone=

```
GET /parcelles?bbox=1.42,43.59,1.46,43.61
```

```
GET /parcelles?zone={"type":"Polygon","coordinates":[[[1.42,43.59],...]]}&mode=intersects
```

La méthode lit les paramètres **dans cet ordre** :

- bbox et zone sont lus et strip(). Les deux à la fois → 400 : on ne devine pas lequel gagne.
- mode : la valeur reçue, sinon within par défaut. Validée par geo.validate_mode() [slide 26]
 - within — **entièrement dedans**
 - intersects — **Une Partie dedans**
- Ni bbox ni zone : un mode tout seul → 400 Sans zone du tout, on rend le queryset **tel quel**.
- geo.read_bbox() [slide 24] ou geo.read_geojson() [slide 25] selon le paramètre présent, puis geo.validate_polygon(.., label="zone") [slide 26].
- geo.in_zone() — le filtre part en base [slide 27], le SQL produit est **au slide suivant**.

Zone : le SQL produit

within — la parcelle est **entièrement dedans** :

```
-- GET /parcelles?bbox=1.42,43.59,1.46,43.61
SELECT id, ..., ST_Area(CAST(geom AS geography)) AS computed_area
FROM parcelle
WHERE ST_Within(geom, ST_GeomFromEWKB('SRID=4326;POLYGON((1.42 43.59,
    1.46 43.59, 1.46 43.61, 1.42 43.61, 1.42 43.59))'))
ORDER BY id LIMIT 200;
```

intersects — la parcelle **déborde mais touche** la zone :

```
-- GET /parcelles?zone={"type":"Polygon",...}&mode=intersects
WHERE ST_Intersects(geom, ST_GeomFromEWKB('SRID=4326;POLYGON((..))'))
```

POST /parcelles — de GeoJSON à INSERT

```
{"type": "Feature",  
  "geometry": {"type": "Polygon", "coordinates": [[[1.4451,43.6042], ...]]},  
  "properties": {"code_insee":"31555","prefixe":"806","section":"AB","numero":"139"}}
```

-- 1. identifiant deja pris ? -> 409

```
SELECT 1 FROM parcelle  
WHERE code_insee='31555' AND prefixe='806'  
      AND section='AB' AND numero='139' LIMIT 1;
```

-- 2. chevauchement ? -> 409 (bbox d'abord, forme ensuite)

```
SELECT id FROM parcelle  
WHERE geom && ST_GeomFromEWKB('SRID=4326;POLYGON((1.4451 43.6042,...))')  
      AND ST_Relate(geom, ST_GeomFromEWKB('..'), 'T*****')  
ORDER BY id;
```

-- 3. rien ne bloque -> on ecrit

```
INSERT INTO parcelle (code_insee, prefixe, section, numero, geom,  
  min_lon, min_lat, max_lon, max_lat, surface_m2)  
VALUES ('31555','806','AB','139', ST_GeomFromEWKB('..'),  
  1.4451, 43.6042, 1.4462, 43.6051, NULL) RETURNING id;
```

ParcelleDetailView

generics.RetrieveUpdateDestroyAPIView : GET, PUT, PATCH, DELETE sur /parcelles/{id}. Le 404 sur un id inconnu est automatique. Le queryset est le même geo.parcelles_with_area() [slide 19], les contrôles d'écriture ceux du POST [slide 21].

```
-- GET /parcelles/4211
SELECT id, ..., ST_Area(CAST(geom AS geography)) AS computed_area
FROM parcelle WHERE id = 4211;

-- PATCH /parcelles/4211 {"geometry": {"type": "Polygon", ...}}
-- les memes 2 controles qu'au POST, en s'excluant soi-meme :
SELECT id FROM parcelle
WHERE geom && ST_GeomFromEWKB('..')
      AND ST_Relate(geom, ST_GeomFromEWKB('..'), 'T*****')
      AND NOT (id = 4211);      -- sinon une parcelle chevaucherait sa version
UPDATE parcelle SET geom = ST_GeomFromEWKB('..'),
      min_lon=.., max_lat=.., surface_m2 = NULL, updated_at = now()
WHERE id = 4211;

-- DELETE /parcelles/4211
DELETE FROM parcelle WHERE id = 4211;      -- puis 204, corps vide
```

GET /parcelles/{id}/voisines

generics.ListAPIView : lecture seule. get_object_or_404 puis geo.neighbours() [slide 28].

-- 1. la parcelle de reference

```
SELECT * FROM parcelle WHERE id = 4211;      -- sinon 404
```

-- 2. celles qui la touchent

```
SELECT id, ..., ST_Area(CAST(geom AS geography)) AS computed_area
FROM parcelle
WHERE ST_Touches(geom, ST_GeomFromEWKB('SRID=4326;POLYGON((...))'))
      AND NOT (id = 4211)
ORDER BY id LIMIT 200;
```

parcelles_with_area()

```
AREA_M2 = Area(Cast("geom", GeometryField(srid=4326, geography=True)))
```

```
def parcelles_with_area():  
    return Parcelle.objects.annotate(computed_area=AREA_M2).order_by("id")
```

```
SELECT ..., ST_Area(CAST(geom AS geography)) AS computed_area  
FROM parcelle ORDER BY id;
```

- **Cast** : geom est stocké en degrés. Sans cast, ST_Area multiplie des degrés (pas de sens) , cast permet de donner le droit exact à PostGIS
- **annotate**, pas une boucle Python : une colonne de plus dans la même requête, pas 200 allers-retours pour une page de 200 parcelles.
- `order_by("id")`
- Appelé par la liste, le détail et les voisins.

validate_polygon() — quatre refus

À ce stade la géométrie n'est plus du JSON : DRF l'a déjà transformée en GEOSGeometry. La même fonction sert pour la parcelle qu'on écrit et pour la zone qui filtre [slide 26].

```
if geom.geom_type != "Polygon": # une parcelle n'est pas un point,
    raise InvalidGeometry(...)

if geom.srid is None: # pas de SRID -> on suppose WGS84
    geom.srid = SRID
elif geom.srid != SRID: # un autre SRID ->
    raise InvalidGeometry(...)

if geom.empty: # un polygone sans point
    raise InvalidGeometry(...)

if not geom.valid: # topologie pas correct
    raise InvalidGeometry(geom.valid_reason) # msg err raison
```

Pourquoi on valide : une géométrie invalide fait planter ST_Relate en pleine requête → 500. Attrapée ici, c'est un 400 avec la raison exacte.

overlaps() — deux étages

```
queryset = (Parcelle.objects
            .filter(geom__bboverlaps=geom)           # filtre 1
            .filter(geom__relate=(geom, OVERLAP_MASK))) # filtre 2 exact

WHERE geom && ST_GeomFromEWKB('..')                -- 8 521 -> ~3 lignes
AND ST_Relate(geom, ST_GeomFromEWKB('..'), 'T*****')
```

- `geom__bboverlaps=geom` : c'est l'index GIST qui répond, donc immédiat — mais il donne des **faux positifs** (deux rectangles se croisent, les polygones non).
- **ST_Relate** compare les **formes réelles**. Coûteux, donc appliqué seulement aux quelques candidats survivants.
- Le masque `'T*****'` [slide 22] — appelé par `validate()`, au **POST** comme au **PATCH**.

'T*****' — ce que dit le masque

Toute géométrie a trois parties : **intérieur** I, **bord** B, **extérieur** E. La matrice croise celles de A et de B — neuf cases, lues de gauche à droite.

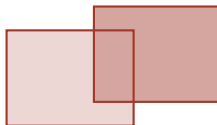
	I(B)	B(B)	E(B)
I(A)	T	*	*
B(A)	*	*	*
E(A)	*	*	*

- Valeurs possibles : F vide, 0 des points, 1 des lignes, 2 une surface, T non vide, * peu importe.
- **Seule la case 1 est contrainte** : l'intérieur de A rencontre l'intérieur de B — les deux parcelles se marchent dessus.
- Ce masque est celui du second filtre de `overlaps()`.

Voisines ou en conflit : la même mécanique



accepté



surface commune → 'T*****'

intérieurs qui se croisent : 409

Pourquoi pas ST_Overlaps T*T***T** ?

Cas	ST_Overlaps	'T*****'
A déborde sur B	vrai	vrai
A entièrement dans B	faux	vrai
A identique à B	faux	vrai
A colle à B	faux	faux

read_bbox() — quatre nombres → un rectangle

```
?bbox=1.42,43.59,1.46,43.61
```

```
-> POLYGON((1.42 43.59, 1.46 43.59, 1.46 43.61, 1.42 43.61, 1.42 43.59))
```

- Pas 4 morceaux → 400
- Pas des nombres → 400
- `min >= max` → 400
- `Polygon.from_bbox()` construit le rectangle, on lui pose le `srid` à la main — GEOS ne le devine pas.
- Appelé par `_filter_by_zone`, puis le rectangle passe par `validate_polygon()`.

read_geojson() — le JSON d'abord

```
try:
    json.loads(text)                # 1. est-ce du JSON ?
except ValueError as error:
    raise InvalidGeometry(f"Unreadable zone: invalid JSON ({error}).")

try:
    return GEOSGeometry(text)       # 2. est-ce une geometrie ?
except (GEOSEException, GDALException, ValueError, TypeError) as error:
    raise InvalidGeometry(f"Unreadable zone: {error}")
```

- Pourquoi vérifier le JSON alors que GEOS lirait la chaîne ? Parce que GEOSGeometry accepte aussi WKT et EWKB. Sans ce contrôle, l'API accepterait en douce trois formats au lieu du seul qu'elle annonce.
- Et une virgule oubliée donne `invalid JSON, ligne 1 colonne 34 :` au lieu d'une erreur de parseur GEOS incompréhensible.
- Appelé par `_filter_by_zone` quand `?zone=` est présent.

La zone : `validate_polygon()` et `validate_mode()`

La zone passe par la même fonction que la parcelle : mêmes quatre refus, un polygone unique en WGS84, non vide, topologiquement correct. Écrire une seconde fonction, c'était recopier les quatre contrôles pour ne changer que les mots des messages.

```
def validate_polygon(geom, label="parcelle"):    # label -> juste le texte
    ...                                           # de l'erreur
```

```
geo.validate_polygon(raw, label="zone")    # -> "A zone is a single polygon; ..."
```

`validate_mode()` — la valeur est interpolée dans un lookup :

```
queryset.filter(**{f"geom_{mode}": zone})    # mode vient de l'URL !
```

- Donc liste blanche ("within", "intersects"), jamais un passe-plat. Une valeur inconnue → 400 avec les valeurs acceptées.
- Sans ce contrôle : `?mode=invalid` ou pire partirait directement dans l'ORM.
- Appelés par `_filter_by_zone`; `validate_polygon()` est la même fonction que pour la parcelle.

in_zone() — une ligne, deux SQL

```
def in_zone(queryset, zone, mode=DEFAULT_ZONE_MODE):  
    return queryset.filter(**{f"geom_{mode}": zone})
```

- mode="within" → ST_Within(geom, zone) — la parcelle est **entièrement dedans**.
- mode="intersects" → ST_Intersects(geom, zone) — elle **déborde mais touche**.
- **Le filtre s'empile** sur le queryset reçu (les filtres d'identifiant sont déjà posés) : **un seul** SELECT au bout.
- **Rien n'est exécuté ici** : un queryset Django est paresseux, la requête part quand la pagination découpe la page.
- Appelé par `_filter_by_zone` — le SQL produit est **ici**.

neighbours() et same_geometry()

```
def neighbours(parcelle):  
    queryset = parcelles_with_area().filter(geom__touches=parcelle.geom)  
    return queryset.exclude(pk=parcelle.pk)
```

- ST_Touches = les bords se rencontrent, les intérieurs non — exactement le contraire de overlaps().
- exclude(pk=...) : une parcelle ne se touche pas elle-même, mais mieux vaut l'écrire que le supposer.
- Le queryset part de parcelles_with_area() : les voisines sortent avec leur surface, comme partout ailleurs.

```
def same_geometry(one, other):  
    return one.equals(other) or one.equals_exact(other, GEOMETRY_TOLERANCE)
```

- Sert au PATCH : la géométrie a-t-elle vraiment changé ?
- Si oui → surface_m2 = None, la contenance officielle ne vaut plus rien.
- La tolérance : 43.60420000000001 et 43.6042, c'est le même point — 0,1 mm sur le terrain, du bruit de flottant.

Bilan : trois idées

- ① **Le calcul reste en base.** Surface, chevauchement, voisinage, filtre de zone : tout part en SQL. Python assemble la requête, PostGIS répond — **jamais de boucle sur 8 521 parcelles.**
- ② **Grossier puis exact.** Le rectangle englobant élimine 99 % des candidats via l'index GIST, la géométrie exacte tranche sur ce qui reste. C'est le motif de `overlaps()`, et c'est ce que fait PostGIS lui-même pour `ST_Touches`.
- ③ **Une seule exception, deux codes HTTP.**
 - `InvalidGeometry` sur une **entrée mal formée** → **400** (géométrie invalide, `bbox` à 3 nombres, `mode` inconnu).
 - `overlaps()` qui **renvoie des lignes** → **409** : l'entrée est correcte, c'est l'état de la base qui refuse.

- **urls.py** — je définis les URLs : quatre `path()` qui associent une route à une vue [slide 11], plus `admin/` branché dans `config/urls.py`.
- **pagination.py** — `FeatureCollectionPagination` découpe les listes en pages de 200 et emballe le résultat en `FeatureCollection` au lieu du `{count, next, previous, results}` de DRF.
 - Déclarée dans `settings.py` (`DEFAULT_PAGINATION_CLASS`), pas dans chaque vue.
 - Donc elle s'applique à tous les GET de liste d'un coup — `/parcelles` comme `/voisines`, sans une ligne à recopier.
 - `?page=2&page_size=50`, plafonné à 1000 : c'est le client qui choisit, dans une limite qu'il ne fixe pas.
- **exceptions.py** — `Conflict`, une exception DRF qui rend un 409 avec un corps structuré (`code, detail, parcelles_en_conflit`) : le client sait quelles parcelles bloquent, pas seulement que ça a échoué [slide 10].

admin

Ce que le projet ne fait pas

- Pas de tests unitaires.
- Pas pret pour le déploiement. Il faudrait Docker (Django + PostGIS), uWSGI comme serveur applicatif et nginx devant (TLS, statiques, timeouts).
- Pas d'authentification.
- Pas de documentation générée
- absence de logs

- **Intégrer un LLM** il produit des paramètres validés par les mêmes contrôles [slide 26], l'API reste seule maîtresse du SQL.
- **Recherche de proximité** : `ST_DWithin` — n'ex les parcelles à moins de 300 m d'ici z